

# RabbitMQ Status Quo Critical Review

Olaf Reitmaier Veracierta - <[olafrv@gmail.com](mailto:olafrv@gmail.com)> - March, 2023

|                                     |    |
|-------------------------------------|----|
| RabbitMQ Status Quo Critical Review | 1  |
| Motivation                          | 2  |
| RabbitMQ Concepts                   | 2  |
| RabbitMQ Architecture               | 3  |
| RabbitMQ Purpose                    | 3  |
| RabbitMQ Well-Known Trade-offs      | 4  |
| RabbitMQ Advantages                 | 5  |
| RabbitMQ Disadvantages              | 5  |
| RabbitMQ Alternatives               | 7  |
| AWS MQ for RabbitMQ (Since 2020)    | 7  |
| Apache Kafka (Since 2011)           | 8  |
| AWS MSK (Since 2018)                | 9  |
| Apache Camel (Since 2007)           | 10 |
| Apache Pulsar (Since 2019)          | 11 |
| AWS SQS/SNS (Since 2006)            | 12 |
| AWS Event Bridge (Since 2019)       | 13 |
| SaaS Low-Code Brokers               | 15 |
| Comparison                          | 16 |
| Conclusions                         | 17 |
| Cost Estimation (AWS)               | 18 |

# Motivation

During the last years many companies have been relying on a self hosted RabbitMQ (RMQ) community version clusters as the central messaging and queueing (MQ) system (or platform) for the landscape of application and services.

RabbitMQ core is implemented in Erlang language (<https://www.erlang.org/>) released in 2007, has a diverse ecosystem of client libraries and a vast and experienced community, and was acquired in 2019 by VMware.

For the last two years, many companies relied on Erlang Solutions, a company offering an enterprise support plan for around €50k/year, the SLA and quality of support is first class level.

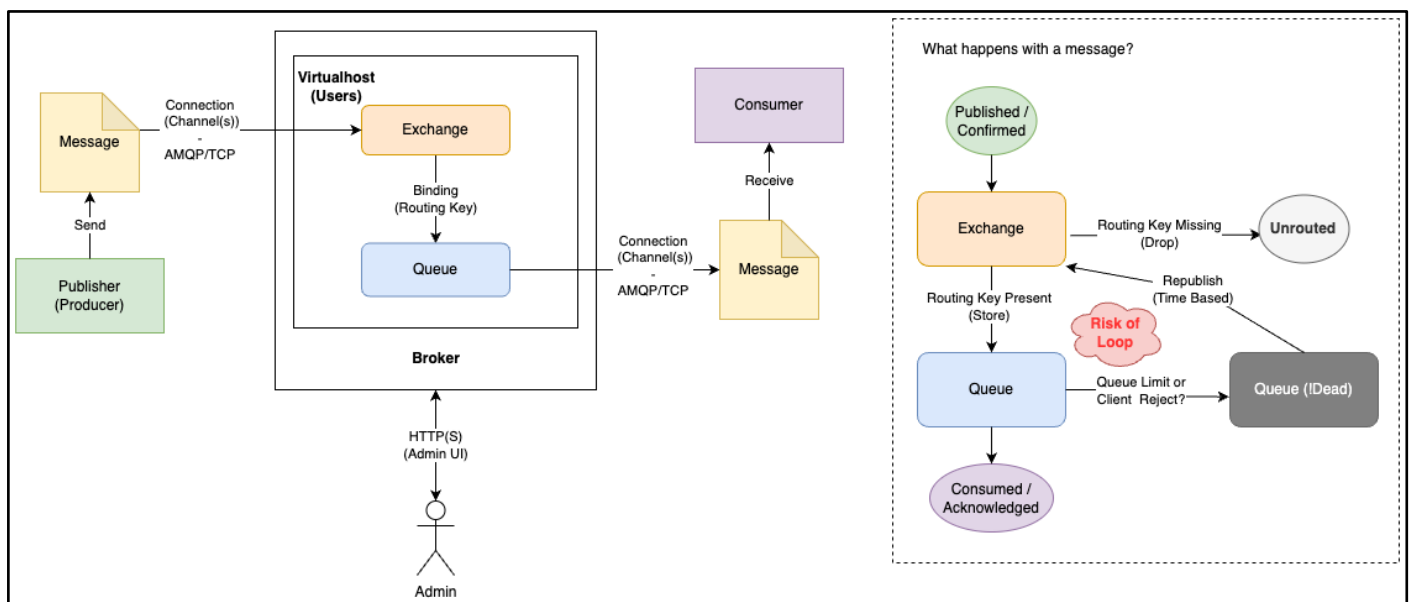
However, as part of the journey to the cloud, in one hand, the switching from self-hosted solutions to managed services (aaS), and in the other hand, related new technologies stacks (e.g. Apache Kafka, Apache Pulsar) and cloud services (e.g. AWS SNS/SQS, AWS Kinesis, AWS Event Bridge) have appeared and were adopted by many companies to migrate or implement brand new MQ system architectures.

Moreover, in the last decade companies evolved progressively through: physical servers, virtual servers, Linux chroot/jails, Linux cgroups, Linux containers (i.e. Docker), Linux container orchestration (e.g. Kubernetes) and Linux serverless (e.g. AWS Lambda). Linux prevails since the 90's, what changed is the way developers interact with it. The same analogy can be applied to MQ systems, and so to RMQ, why can not RMQ still be around like Linux is?

The purpose of this document is to revisit the current RMQ status quo and give a concise overview of advantages, disadvantages, bust the myths, and determine if still is the proper solution or not, considering others alternatives that seem to fit companies MQ system future requirements.

## RabbitMQ Concepts

Basic concepts to comprehend the jargon of RMQ (and most of MQ systems) are explained in a comprehensive way in the following article <https://www.rabbitmq.com/documentation.html> and is important to understand them to continue through the rest of the document. However, for the inpatients a summary follows.



In RMQ a set of users, connections/channels, exchanges, queues and policies is grouped into a virtual host (vhost).

Clients connects to RMQ using AMQP TCP-based protocol (<https://www.amqp.org/>), like TCP is a well-known protocol not exclusive of RMQ and adopted by many MQ systems and companies (<https://www.amqp.org/about/examples>), including but not limited to: Apache Qpid, SwiftMQ, JORAM, Microsoft Azure Service Bus, StormMQ and MQLight.

Messages have an agreed but not enforced standard JSON schema, while being published/confirmed by publishers (aka. senders/origins) and received/acknowledged/rejected by consumers (aka. receivers/destinations).

RMQ stores messages in queues temporarily on memory and/or persistent local disks. Queues can be accessed via a construct called exchange which is in charge of user authentication, connection pooling (channels), message binding (routing) to queues and policing (e.g. TTL, size, limits, etc).

## RabbitMQ Architecture

In general, a lot of companies rely on cloud services to enable servers for RabbitMQ. In this document I will focus on a very common RMQ architecture on top of AWS services which I have seen in many companies over the last years:

- AWS EC2 based RabbitMQ clusters.
- Separated clusters for each environment: staging, integration and production.
- Separated clusters per environments for: business messages and log streaming.
- Cluster are running in an specific AWS Account, but each environment on different VPCs.
- RMQ individual nodes are spread evenly across three (3) different AWS availability zones (AZ).
- Each application/service publishes/consumes messages within its own or to/from other virtual hosts (vhost). There is one vhost for each application/service group and a couple of “global” vhosts used to broadcast messages to selected or all other existing vhosts. At low level, publish/consume operations are done against exchanges which are tied to specific queues by a binding (routing) key.
- Publish/Consume cluster endpoints for AMQP clients could (or not) have TLS enabled.
- AMQP client connections are load balanced via DNS Round-Robin or AWS load balancer.
- RMQ Admin Web interface endpoints are behind AWS Load Balancers with TLS enabled.
- User authentication is managed locally by RMQ without any Single Sign On (SSO) integration.
- RMQ is monitored from a Prometheus/Grafana Stack calling the cluster admin API.

## RabbitMQ Purpose

Currently, many companies uses RMQ for the following main purposes:

- Decoupling: RMQ avoids direct access from origins outside the destination software domain. For example, domain A needs to read data from the database of the domain B, there are several options: A calls directly API of B (if API is available), A query the B database directly (Coupling), or A queues a RMQ message request to B, then B queues a response message to A (Decoupling).
- Buffering: RMQ absorbs direct load that will cause Denial of Service (DoS) if it hits directly an application, service or database when the target of the source requests has a low throughput or slow speed, otherwise the destination would require instantaneous scaling or throttling mechanism to cope with the requests.
- Business Messages Exchanging: RMQ is used as a broker for exchanging business data as messages within the same or different software domain(s). From now on, when there is a reference to “business data” is considered different from “log data”, which is used for debugging, monitoring and alerting.
- Monitoring Messages Streaming: RMQ is used as a broker for streaming log records in JSON and plain text format, produced by web servers (mostly NGINX) and applications/service workers/functions. Messages are finally streamed to different stores (i.e. AWS OpenSearch, S3 CloudWatch). Some companies started to replace this use case with Vector (<https://vector.dev/>) or SaaS solutions. Vector is a modern metric and logs streaming solution based on “observability pipelines” (it is not a MQ system).

# RabbitMQ Well-Known Trade-offs

Before talking about advantages and disadvantages of RMQ in the coming sections, it is important to reflect objectively on things that are **NOT disadvantages of RMQ** itself or any other MQ system, instead they are trade-offs.

Actually, MQ cloud based systems can be deployed as IaaS like RMQ or PaaS like Amazon MQ for RabbitMQ both with "virtual fixed" capacity. However, they can also be just SaaS with "unlimited virtual" capacity like AWS SQS/SNS.

**IaaS/PaaS MQ systems are better for high performance** requirements and scale mostly vertically. In this case, cluster architectures were introduced for high availability not massive horizontal scaling. On the contrary, **SaaS MQ systems are better for massive scalability** requirements and scale mostly horizontally.

However, I have learned over the years, that **any system architecture** (including MQ systems), do not escape from the following assertion that I like to say "the **safety of any system** is based on **two rules**":

- **If the system capacity is limited** then the **clients must implement throttling** to avoid Denial of Service (DoS) scenarios, this always requires a holistic coordination effort between clients. Developers never have time or forget about it, blaming the system for the undesired scenarios.
- **If the system capacity is unlimited** then the **system must implement throttling** and not trust in the good will of the clients (developers), this always requires a defensive design and implementation. Cloud Service Providers like AWS have throttling actions triggered on every SaaS (aka serverless) service based on predefined quota/limits. So, they force "cloud-native" refactoring of clients and avoid costs or resource waste, considering that the cloud has a virtual elastic capacity but physically still its capacity is fixed.

Evidence (from my experience) shows that frequently the effects of the following practices are underestimated:

- **Unexpected or Indiscriminate broadcasting**, even to destinations just discarding messages.
- **Too fast message publication** (lack of throttling) + **too slow consumption** (lack of scaling). This means that those clients do not implement confirmation while publishing or acknowledgement while receiving messages, or any other custom rate limit mechanism.
- **Huge message sizes** which is a questionable processing "mainframe-like" system pattern.
- Usage of **old RMQ client libraries** not supporting heartbeats, or not using heartbeats (<https://www.rabbitmq.com/heartbeats.html>) or **not coding a connection recovery** mechanism (<https://docs.aws.amazon.com/amazon-mq/latest/developer-guide/best-practices-rabbitmq.html#best-practices-rabbitmq-connection-recovery>) when using the new RMQ client libraries.
- **Lack of message schema checks** at consumer/publisher start time. Instead some companies have implemented validation tools/pipelines for debugging (good) and post-mortem (bad, too much resource waste). However, this approach is not correct because it resembles checking all SQL INSERTS in real-time to ensure proper TABLE SCHEMA is used.
- **Creating messages (or queues) and NOT consuming (using)** them for days, weeks or months, considering RMQ as a long term storage alternative.
- **Lack of alert thresholds** for message publish / consume rates for critical queues.
- **Log shipping via RMQ**. Currently, many companies still are using RMQ to ship logs. Developers must stop using RabbitMQ for log shipping. For example, Vector has buffering, routing and transformations capabilities not possible in RMQ. The Vector approach is that application/services just write logs to local files and forget about the rest.

The **impact in RMQ is the same** - a denial of service (DoS) within seconds/minutes - with the following symptoms:

- Message routing loops burning server CPU.
- Exhaustion of network bandwidth and/or server memory.

- RMQ throttling or pausing, when the underlying Linux OS or when the cloud provider quotas/limit are exceeded (e.g. EC2 Network plateau in CloudWatch for EC2 CPU/Network metrics).

In the worst cases, RMQ system components become unresponsive, locked or hung, requiring **manual intervention** when throttling or rate limits mechanisms are triggered by RMQ itself.

Effects are so bad and sudden that either stopping the culprit or restarting the RMQ affected component is the only and faster way to fix, any auto (or reactive) scaling mechanism (manoeuvre) render useless.

This situation **prevents downscaling** as other platforms and forces RMQ to deal with an **internal DoS situation**.

It is clear that the **same movie ending will come to any self-hosted or cloud managed MQ system** under such pressured scenarios and high stress conditions, or will translate into a waste of resources (money).

Hence, **refactoring applications logic or implementing code fixes** for issues described before is a precondition to improve the resilience not only of RMQ but of any managed MQ system.

## RabbitMQ Advantages

- Simplicity:
  - In General, RMQ is very simple, it uses a message-queue-routing model. In 10 minutes you can set up what is needed or go with defaults (i.e. users, vhost, exchanges, routing, routing (binding) keys, queues) and right away start publishing and consuming messages in a few lines of code in your favourite programming language. If you won't believe me, just go to <https://www.cloudamqp.com/plans.html> register for a free plan or do docker pull [https://hub.docker.com/\\_/rabbitmq](https://hub.docker.com/_/rabbitmq), then just follow this beginners tutorial: <https://www.rabbitmq.com/tutorials/tutorial-one-python.html>.
- Performance:
  - RMQ has proven to be the best MQ high performance system for years beside the abuse against it that is still underestimated. It can scale vertically to huge CPU/Memory levels, and now in AWS with better hardware CPUs feels better compared to on-premises setups.
- Reliability:
  - RMQ clusters in the data centre and later in AWS have been very reliable besides the abuse described in the well-known trade-off section.
- Maturity:
  - RMQ supports AMQP, STOMP and MQTT protocols. It has client libraries on all the major programming languages: <https://www.rabbitmq.com/devtools.html>, some are natively certified and supported, including Java, JavaScript, Python, Go, PHP and Rust. It has been here since 2007.
- Cost:
  - Generally, people use RMQ community edition, so there are no licensing costs.
  - Enterprise support from Erlang Solutions for around €50k/year, which includes quarterly assessments (health checks), excellent response SLA and quality of expertise.

## RabbitMQ Disadvantages

- Fault Tolerance:
  - Lack of network load balancing for AMQP clients, when using non-certified libraries (e.g. old PHP library) plus AWS load balancer service limitations:
    - The old PHP amqp-lib C-based library (<https://github.com/php-amqplib/php-amqplib>) widely used to implement RMQ clients is not able to spawn asynchronous heartbeat thread (PHP is single threaded) to check RMQ connection liveness.
    - The lack of heartbeats or recovery mechanism and the decision of AWS to not reply with RST packets to clients behind an AWS NLB (Network Load Balancer), provokes that RMQ clients get stuck, especially during RMQ node failover. Also I found that this connection failover issue still exists when using AWS

MQ for RabbitMQ (<https://docs.aws.amazon.com/amazon-mq/latest/developer-guide/best-practices-rabbitmq.html#best-practices-rabbitmq-connection-recovery>).

- Many migrated RabbitMQ clusters to AWS but were not able to have cloud native load balancing to ensure high availability and fault tolerance for RMQ clients. Subsequently, others started to look for an alternative php-amqp library (<https://github.com/php-amqplib/php-amqplib>) as it was able to support heartbeats (and connection recovery), but many still relies on load balancing via legacy DNS Round-Robin or additional HAProxy setups to load balance connections.

- Scalability (Horizontal):

- RMQ works the best in single node architecture but then becomes a single point of failure, as confirmed by Erlang Solutions.
- RMQ works very well in 3 node cluster architectures (up to 7 is possible). However, performance drops 1/N due to the replication of messages when nodes are added, so more CPU/Network capacity is required to compensate. In recent versions of RMQ, quorum-queues were introduced to reduce the impact on clusters compared to classic/mirror queues. Migrating to the new quorum queues requires the recreation of queues, so it is complex and downtime is expected. This is a pain for RMQ administrators.

- Security:

- The absence of network load balancing for AMQP clients through AWS NLB, results in the lack of an endpoint where TLS termination can be offloaded. As a result, communication occurs in plain text, potentially exposing sensitive information, including user credentials.
- Shared client credentials. In 2021, RMQ announced support for OAuth but many never went for it, due to concerns raised about adding an unnecessary single point of failure for such a critical infrastructure.

- Upgrades:

- For those using RMQ version 3.9.27. RMQ major version 3.9 will be “End of Life” in July 2023. As of the moment of writing this document, upgrading to the latest major version v3.11 is blocked by the deprecation of classic queues in version 3.10 (<https://www.rabbitmq.com/ha.html>) a feature that many still rely on. Also, upgrading to the latest minor version v3.9.\* was recently blocked because contain bugs - *one of them in the administration UI* - that is only fixed in the latest major version 3.11 as confirmed by Erlang Solutions (<https://github.com/rabbitmq/rabbitmq-server/issues/7425#issuecomment-1444875067>).

- Support:

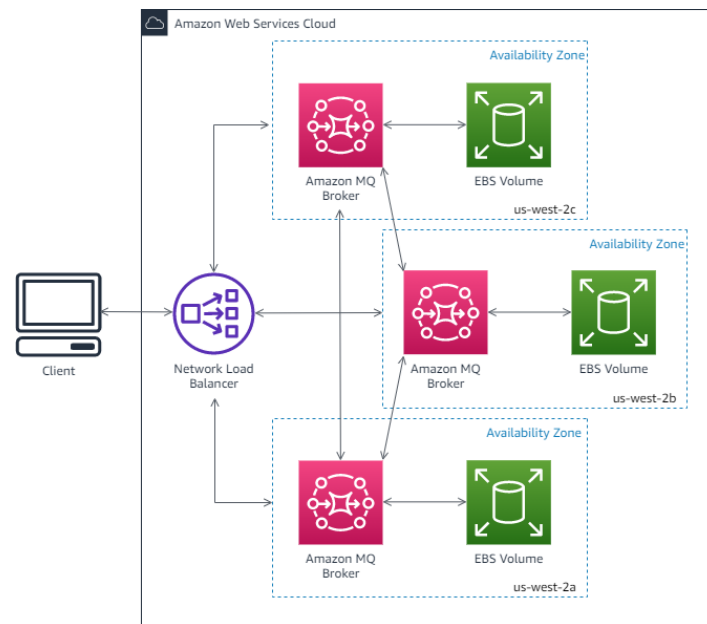
- Although all most developers can work with RMQ and administrators employ acceptable effort to maintain RMQ clusters, only a reduced set of engineers are trained and capable of deeply understanding and managing the RMQ clusters. If you hit a bug or had faced an issue with undetermined cause (although rarely), reaching the community is definitely not enough, and it have been demonstrated that having the Erlang Solution enterprise support contract provides a better outcome for those situations, apart from all the assessment and important improvements derived from their health check reports.

# RabbitMQ Alternatives

The main reason to move away from RMQ is still foggy, but looks like the main driver is to make life even easier for engineers regarding MQ systems, but this will not refrain developer from tackling the issues that arise due to the well-known trade-offs, no matter what course of action is decided; with or without RMQ.

In the rest of the document, I will highlight key facts of the RMQ contenders offering different ways to solve the MQ architectural and implementation challenges, comparing them side-by-side with RMQ, enriching it with the documented experience from adopters of each alternative technology.

## AWS MQ for RabbitMQ (Since 2020)



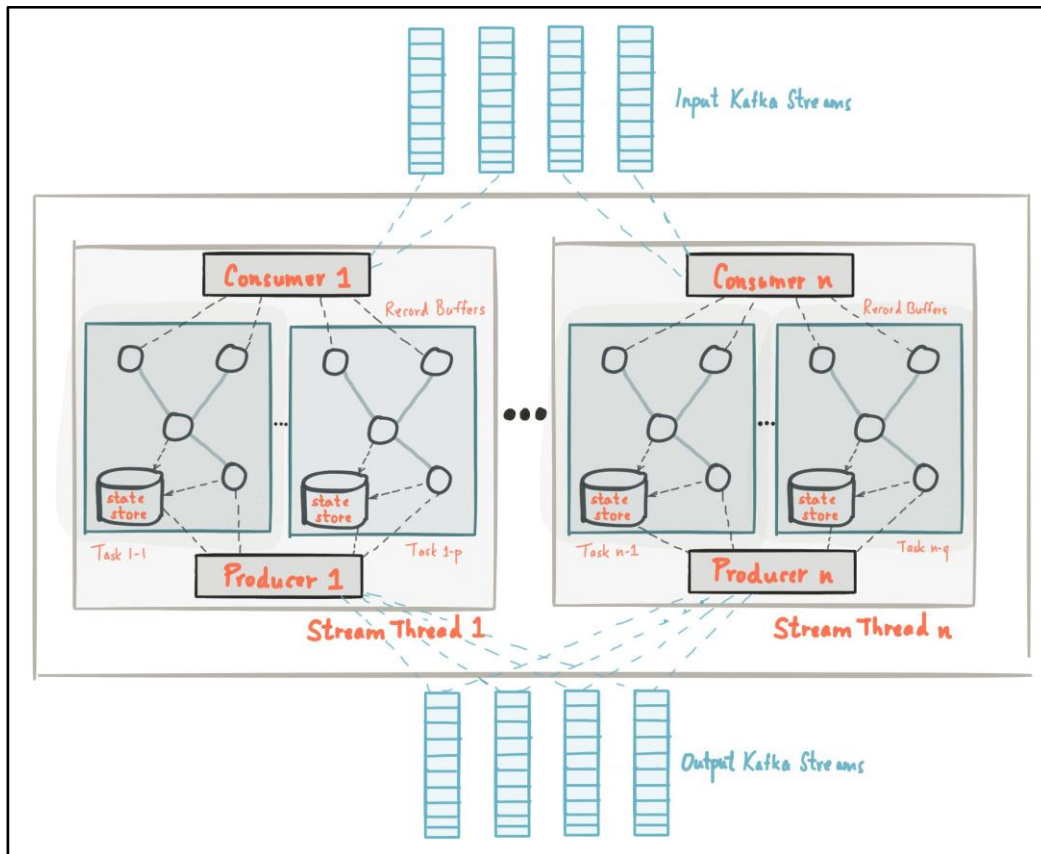
- MQ means Managed Queues or Message Queues?, it is just a fancy AWS registered trademark.
- It is RabbitMQ without the server maintenance burden, with the rest of RMQ mentioned disadvantages, no access to command line (alternative when UI hangs), and fully integrated with AWS satellite services.
- It requires creation of IAM Users with temporary or longer-term credentials, or to introduce SSO - *remember is a single point of failure or another moving part* - for such critical infrastructure.
- It has a hard limit on monitoring (Max. 500 metrics) via CloudWatch, RMQ setups has no limits.
- Current monitoring/alerting has to be reworked from Prometheus Stack to AWS CloudWatch to an unknown extent, a similar thing already happened with AWS RDS Aurora a lot of metrics are not available anymore or are accessible via twisted API calls.
- Unfortunately, there are major blockers to go for it:
  - AWS warns about keeping queues short! and avoid sending unnecessary messages! to avoid hitting unexpectedly their hard limits and quotas, rendering the service unresponsive.
  - AWS warns about the need of libraries with heartbeats / connection retries due to lack of auto network failover, which means that current RMQ clients' open issues with AWS NLB still persist when switching to AWS MQ for RabbitMQ.
  - The maximum instance size is mq.m5.4xlarge with 16 CPUs and 64 GB RAM and "High" network throughput, this is a very bad limitation, as currently 4xlarge is the minimum RMQ instance size in many production setups.

### References:

- <https://docs.aws.amazon.com/amazon-mq/latest/developer-guide/welcome.html>
- <https://docs.aws.amazon.com/amazon-mq/latest/developer-guide/amazon-mq-setting-up.html>
- <https://docs.aws.amazon.com/amazon-mq/latest/developer-guide/best-practices-rabbitmq.html>
- <https://docs.aws.amazon.com/amazon-mq/latest/developer-guide/amazon-mq-rabbitmq-limits.html>



## Apache Kafka (Since 2011)



- Initially developed in Java by LinkedIn.
- It's similar to the Apache Flink (AWS Kinesis) engine.
- It's a cluster of clusters. Uses Apache Zookeeper as a registry.
- Uses publish-subscribe topic-subscription model, while RMQ uses message-queue-routing model.
- Kafka SDK client libraries are targeting the Java audience mostly. For example the PHP libraries have their last commit about +5 years old and there is no official designated library. The recommended client list is available at: <https://cwiki.apache.org/confluence/display/kafka/clients>.
- It relies on its Kafka binary protocol which requires additional work or custom integrations to connect systems that don't natively support it.
- Has plenty of connectors to ingest and deliver data considering pub/sub streams architecture.
- Has a replay feature making it easier to republish messages from an archive (not possible in RMQ).
- Supports real-time message transformations (one of the reasons Kafka exists), not possible in RMQ.
- Has a schema registry (optional), but validation has to be implemented on the client side.
- Has a fault-tolerance mechanism that stores messages in a distributed commit log on disk. This is very advantageous to implement any long term messages retention (archive).
- It is known for its scalability, fault-tolerance, and high throughput, but also introduces additional complexity for developers and administrators. Kafka requires managing topics, partitions, offsets, and consumer group coordination, which may require more effort in configuration and understanding compared to RabbitMQ's more simple queuing and right sizing model (See AWS MSK right sizing complexity).

### References:

- <https://kafka.apache.org/protocol.html>
- <https://kafka.apache.org/11/documentation/streams/architecture>
- [https://docs.confluent.io/platform/current/connect/kafka\\_connectors.html](https://docs.confluent.io/platform/current/connect/kafka_connectors.html)
- <https://tech.willhaben.at/kafka-connect-custom-single-message-transform-using-jslt-2fc57ae98395>
- <https://debezium.io/documentation/reference/stable/transformations/index.html>
- <https://acloudguru.com/hands-on-labs/using-schema-registry-in-a-kafka-application>



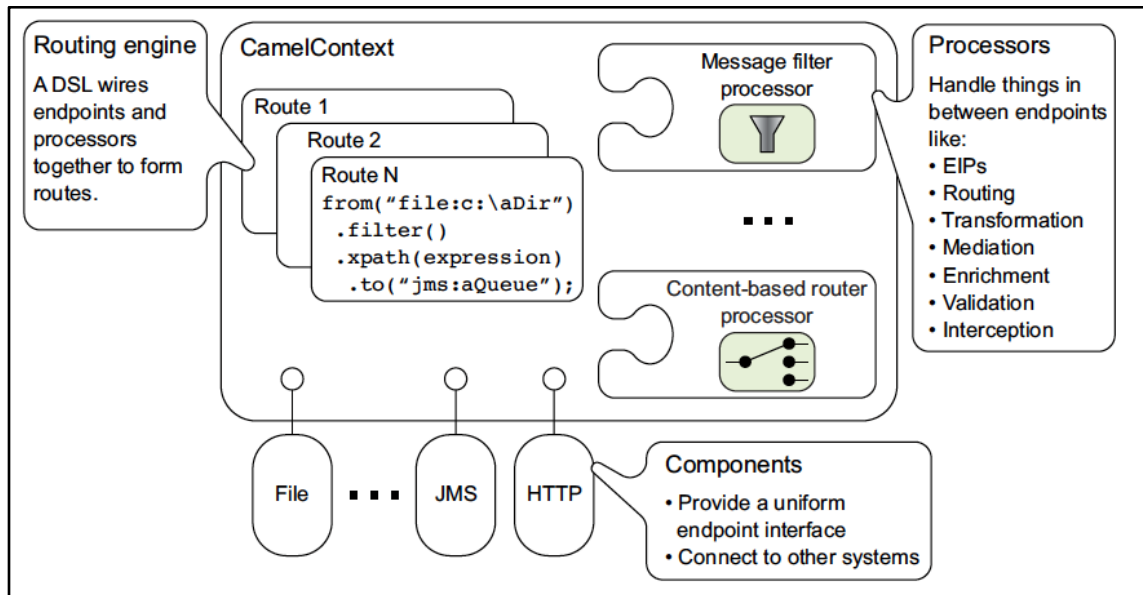
## AWS MSK (Since 2018)

- MSK means Managed Service for Kafka, it is just another fancy AWS registered trademark.
- It is Kafka without server maintenance burden and easy clustering (like AWS RDS Aurora).
- Many Business Intelligence and Analytics Team are using it (but only them), especially to pull data from databases, process it and push it to data warehouses / lakes.
- Right sizing of Kafka self-managed clusters or AWS MSK clusters are both key for stability and it is not as simple as in RMQ clusters, because it requires considering way more variables: [https://view.officeapps.live.com/op/view.aspx?src=https%3A%2F%2Fdy7oqpxkwhskb.cloudfront.net%2FMSK\\_Sizing\\_Pricing.xlsx&wdOrigin=BROWSELINK](https://view.officeapps.live.com/op/view.aspx?src=https%3A%2F%2Fdy7oqpxkwhskb.cloudfront.net%2FMSK_Sizing_Pricing.xlsx&wdOrigin=BROWSELINK) (backlinked from AWS guides).
- There is an offering for AWS MSK Serverless that abstract the complex operation of centralised Kafka, basically is Kafka without the clustering maintenance burden. However, the AWS MSK serverless has very odd limitations to max. 1000 client connections and 15k req/sec, majority of RMQ setups sustain way more demand during normal daily operation.

### References:

- <https://docs.aws.amazon.com/msk/latest/developerguide/before-you-begin.html>
- <https://docs.aws.amazon.com/msk/latest/developerguide/serverless.html>
- <https://docs.aws.amazon.com/msk/latest/developerguide/limits.html>
- <https://docs.aws.amazon.com/msk/latest/developerguide/bestpractices.html>
- <https://aws.amazon.com/blogs/big-data/best-practices-for-right-sizing-your-apache-kafka-clusters-to-optimize-performance-and-cost/>

## Apache Camel (Since 2007)

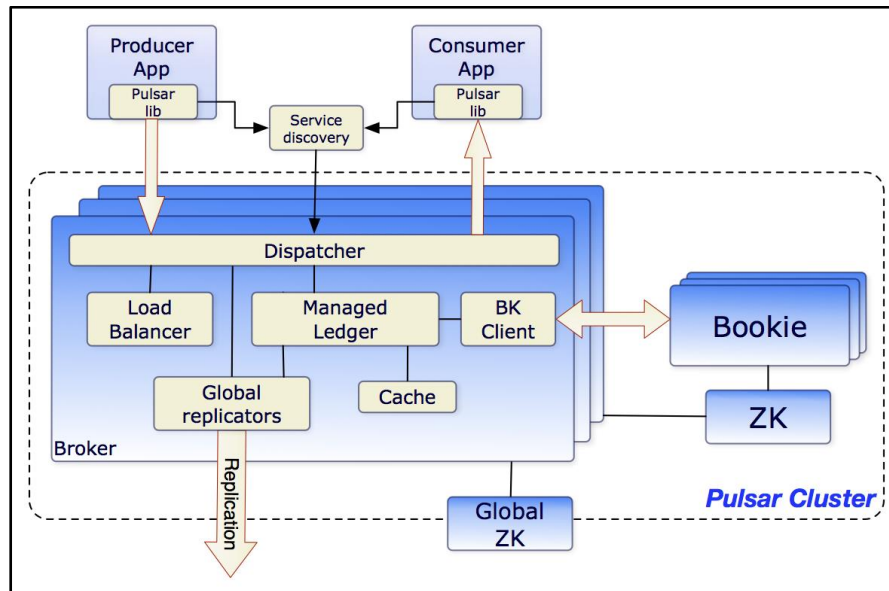


- It is a Java/Bean/Tomcat/Spring/Maven/XML based clusterable integration framework of diverse systems using a variety of protocols (including AMQP) and data formats.
- Has a very steep learning curve due its complex architecture and patterns, a lot of components and documentation really targeting hardcore Java fans. It is indeed harder than Kafka.
- Has all the drawbacks of old Java Enterprise Architecture design (search in Google for "TOGAF").
- Camel clustering and scalability is not one of its strengths, literally.
- Who uses Apache Camel? Well: myself in past projects (i.e. SAP HANA), Apache Foundation (Advisor), RedHat Fuse, JBoss, Netflix (Payment Gateway and others), SAP HANA (Multi-Database Connector), Platform6 (B2B layer development and operationalization).
- Supports real-time message transformations, not available in RMQ.
- I personally won't recommend Apache Camel to company nowadays, unless they do some kind of flight traffic or blockchain system related projects.

### References:

- <https://camel.apache.org/manual/architecture.html>
- <https://camel.apache.org/components/3.20.x/index.html>
- <https://camel.apache.org/components/3.20.x/eips/transform-eip.html>
- [https://help.sap.com/docs/HANA\\_SMART\\_DATA\\_INTEGRATION/7952ef28a6914997abc01745fef1b607/598cd48941a41128751892fe68393f4.html](https://help.sap.com/docs/HANA_SMART_DATA_INTEGRATION/7952ef28a6914997abc01745fef1b607/598cd48941a41128751892fe68393f4.html)
- [https://access.redhat.com/documentation/en-us/red\\_hat\\_fuse/7.5/html/apache\\_camel\\_development\\_guide/index](https://access.redhat.com/documentation/en-us/red_hat_fuse/7.5/html/apache_camel_development_guide/index)
- <https://developers.redhat.com/articles/2021/09/21/distributed-transaction-patterns-microservices-compared>
- <https://camel.apache.org/components/2.x/others/spring-cloud-netflix.html>
- <https://camel.apache.org/community/user-stories/>
- <https://artofcode.wordpress.com/2018/07/31/apache-camel-sucks/>

## Apache Pulsar (Since 2019)

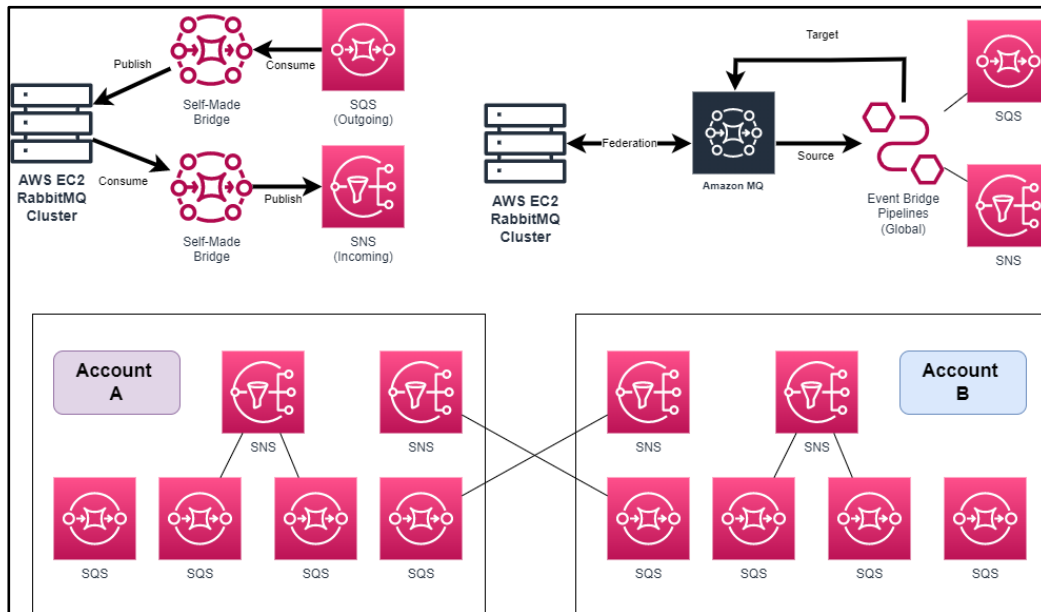


- Very brand new, really promising features, similar to Kafka + AWS Kinesis.
- As Kafka, is a cluster of clusters, with easy initial setup but its observability “looks” a bit complex.
- It uses publish-subscribe topic-subscription model, while in RMQ rely on message routing.
- It exposes a REST API (learning needed) - *not a crazy binary protocol* - similar to AWS services.
- Very simple installation and configuration even for clustering scaling (one liners).
- Connectors / Plugins are growing as part of the plug-able core layer. AWS SQS as destination is missing, but being implemented by a 3rd party, most probably will be included soon in the core plugin list (<https://github.com/streamnative/pulsar-io-sqs/blob/master/docs/sqs-sink.md>).
- Uses Apache Zookeeper cluster as registry as Kafka, but in Pulsar storage is decentralised in “bookies” handled by Apache BookKeeper. No single point of failure or bottleneck when storing messages, while supporting in-memory and persistent storage with custom retention.
- Has a schema registry with real-time validation (goodbye to Mercury and validation pipelines).
- Has an internal proxy, so no HAProxy or external load balancer is needed.
- Can be deployed in Kubernetes clusters, so it scales up and out flawlessly.
- It is multi-tenant via namespaces (like RMQ vhosts but a bit more complex/better).
- Allows message transformations via Java/Python/Go functions, similar to Apache Kafka, AWS Kinesis or AWS Lambda (integrations). There is no need for another FaaS solution (e.g. Apache Storm/Heron/Flink). This is not available in RMQ.
- Implements throttling per broker, topic or subscription. Clients can check in real-time their quotas and speed up or down (sleep). Others does not provides this facility to clients, that is why complex additional monitoring is needed (i.e. NewRelic, Prometheus, Grafana, Kibana, CloudWatch, etc).
- Has statistics and metrics per broker, internal components, topics, consumers and producers.
- Supports client authentication via JWT, OAuth2.0, OpenID, etc. and permissions via ACLs.
- Has built-in geo replication across regions (like RMQ federation but better).
- Streamnative.io is using and supporting it for several big companies (i.e. NetData, Iterable, Microfocus) and Pand.io is offering it as SaaS on the AWS marketplace.
- No cloud vendor lock in.

## References:

- <https://pulsar.apache.org/docs/3.0.x/> (Everything is well written there and improving).
- <https://streamnative.io/deployment/byoc>
- <https://pandio.com/apache-pulsar-as-a-service/>
- <https://aws.amazon.com/marketplace/pp/prodview-o7h4jiwm43vi6>
- <https://pulsar.apache.org/docs/3.0.x/deploy-aws/>
- <https://pulsar.apache.org/docs/3.0.x/cookbooks-retention-expiry/>
- <https://pulsar.apache.org/docs/3.0.x/administration-zk-bk/>
- <https://streamnative.io/blog/how-apache-pulsar-is-helping-iterable-scale-its-customer-engagement-platform>
- <https://streamnative.io/success-stories/how-apache-pulsar-helping-iterable-scale-its-customer-engagement-platform>
- <https://github.com/streamnative/pulsar-io-sqs/blob/master/docs/sqs-sink.md>

## AWS SQS/SNS (Since 2006)

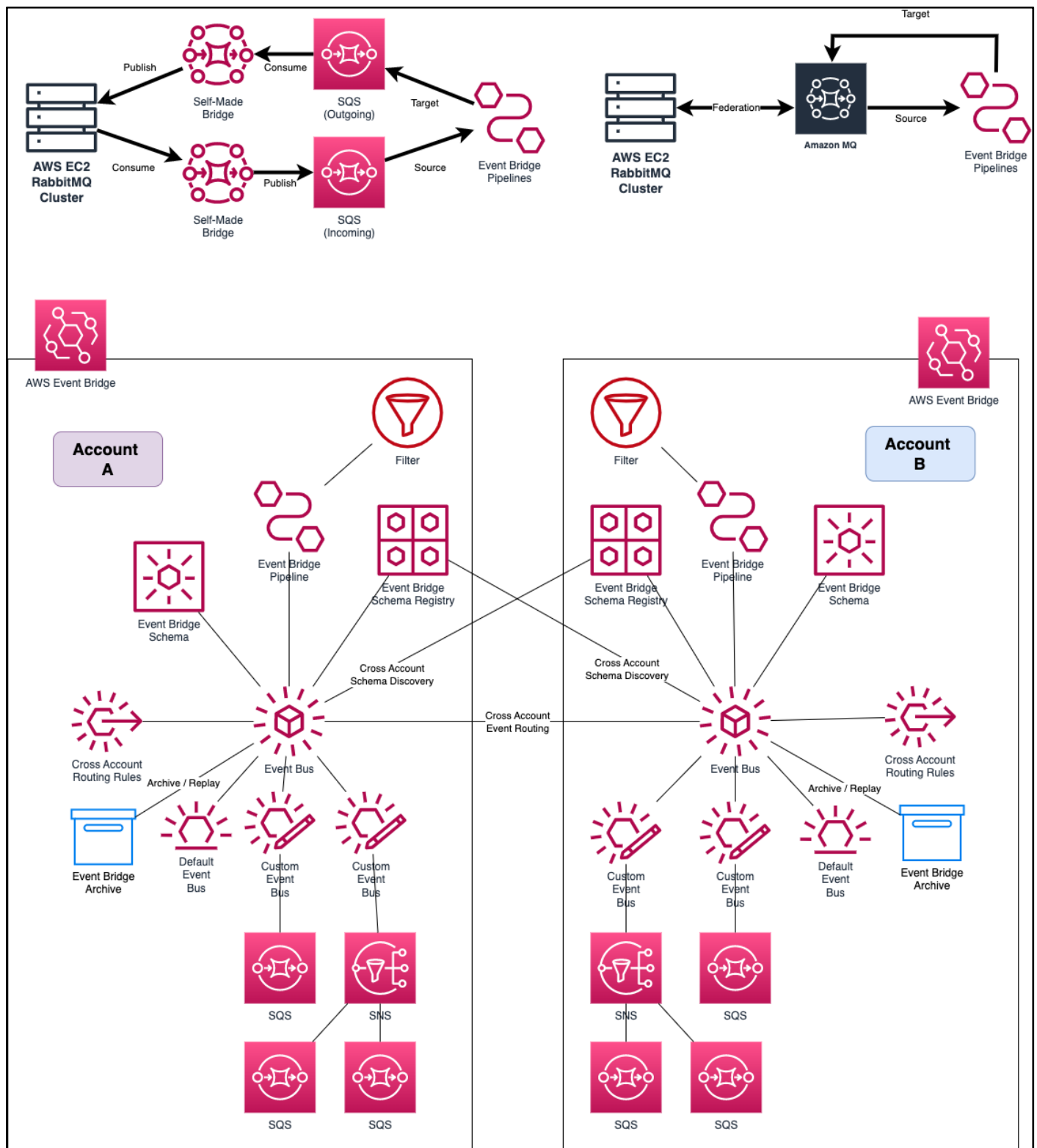


- It was invented before RMQ, so is even older than RabbitMQ.
- It is used across all Amazon internal and public systems for the last 15 years.
- It is a publish-subscribe topic-subscription model, while in RMQ rely on message routing.
- SNS provides a way to forward messages between SQS queues (SQS -> SNS topic subscription).
- Both SQS and SNS are intended for new applications to have unlimited scalability and simple APIs.
- As almost all AWS services, rely on AWS IAM Roles, so no need for shared credentials.
- Compared to RabbitMQ, SQS has a lot of limitations for scale up imposed via hard limit/quotas:
  - A message can live in the queue for up to 14 days.
  - Maximum message size is 256KiB, so payload has to be offloaded to e.g. S3.
  - Maximum 300 transactions (send/receive/delete message) per second/API-call. Maximum 3000 batches transactions (send/receive/delete message) per second/API-call (each batch includes 10 messages). For FIFO queues these limits are doubled, but a batch is still 10.
  - Although SNS is more flexible with a maximum message size of 2 GB (bottleneck).
  - Message polling calls without messages are also counted as API-calls.
  - Several of these limits are exceeded globally on many RabbitMQ setups, so further investigation on publishers / consumers refactoring checks are needed (e.g. batching).
- Cross account AWS IAM security architecture can become a nightmare for having control and observability centralised for all accounts (Terraform?).
- It offers a simple and useful interface to sampling messages, but the monitoring features are buggy and CloudWatch metrics have a huge lag, not comparable with the 60/sec RMQ/Prometheus stats.
- Monitoring and logging is reduced to what AWS CloudWatch offers for the aforementioned services.
- All systems will be in AWS cloud vendor lock in.

### References:

- <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-basic-architecture.html>
- <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-queue-types.html>
- <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-quotas.html>
- <https://docs.aws.amazon.com/sns/latest/dg/sns-event-sources.html>
- <https://docs.aws.amazon.com/sns/latest/dg/sns-event-destinations.html>
- <https://docs.aws.amazon.com/sns/latest/dg/large-message-payloads.html>
- <https://aws.amazon.com/blogs/compute/cross-account-integration-with-amazon-sns/>
- <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-difference-from-amazon-mq-sns.html>
- <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/reducing-costs.html> (Reducing Costs)

## AWS Event Bridge (Since 2019)



- AWS CloudWatch Events, announced in 2016, was merged into Event Bridge in 2019.
- AWS Event Bridge offers routing for AWS SQS/SNS (and many other (non-)AWS services).
- In SQS events (messages) can be processed one by one or in batches and deleted after successful processing, while in the Event Bus each message is processed one by one and can match multiple rules and sent to multiple targets, processing ends when there are no rules pending.
- Has a schema registry, but validation has to be done on the client side (better), or implemented itself as a transformation (e.g. AWS Lambda) that will increase costs.

- It supports sourcing events from different AWS services including Amazon MQ for RabbitMQ or SNS/SQS into an AWS Event Bridge Pipe(line), this could be useful for a federation (limited scalability of Amazon MQ) or in-house made bridge with RMQ clusters.
- AWS Bridge Pipe(lines) allows message routing from/to AWS services, based on filters on attributes of the messages. But the feature is not mature, has some inconsistencies (messages end up in a limbo), poor error/failure tracing and a huge lack of metrics update on CloudWatch widgets/dashboards overall, not comparable with RMQ / Prometheus.
- It supports real-time message transformation, not possible in RMQ.
- It supports message replay like Kafka/Pulsar from Archive, not possible in RMQ.
- It supports AWS multi accounts via custom account Event Bus (default one is for AWS services).
- It provides control over AWS IAM security standards for multi-account designs and architectures.
- AWS Event Bridge has quotas/limits, the default soft values are very low, and the hard limit values are theoretically unlimited. Increasing the soft defaults (throttling) values requires opening an AWS support ticket (per account). Therefore, any unpredicted behaviour can throttle clients (consumer or publishers), even worse than the CPU/Memory/Network issues when RMQ is abused. A blueprint pilot will be needed to determine what are the required limits (if any).
- Monitoring and logging is reduced to what AWS CloudTrail (API) and limited CloudWatch (Metrics) offer for the aforementioned services, compared to Grafana/Prometheus setups.
- All systems will be in AWS cloud vendor lock in, also the previous SNS/SQS cost warnings apply.

#### References:

- <https://aws.amazon.com/eventbridge/>
- <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-pipes-create.html>
- <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-pipes-event-source.html>
- <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-pipes-mq.html>
- <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-pipes-event-target.html>
- <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-cross-account.html>
- <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-event-bus.html>
- <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-quota.html>
- <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-archive-event.html>
- <https://aws.amazon.com/eventbridge/pricing/>
- <https://aws.amazon.com/blogs/compute/introducing-amazon-eventbridge-scheduler/>
- <https://dev.to/aws-builders/should-we-consider-migrate-to-amazon-eventbridge-from-amazon-sns-sqs--4dgi>
- <https://aws.amazon.com/blogs/compute/reducing-custom-code-by-using-advanced-rules-in-amazon-eventbridge/>
- <https://aws.amazon.com/about-aws/whats-new/2021/09/cross-account-discovery-amazon-eventbridge-schema/>
- <https://theburningmonk.com/2023/02/the-biggest-problem-with-eventbridge-scheduler-and-how-to-fix-it/>
- <https://aws.amazon.com/blogs/aws/new-cloudwatch-events-track-and-respond-to-changes-to-your-aws-resources/>
- <https://aws.amazon.com/about-aws/whats-new/2019/07/introducing-amazon-eventbridge/>
- <https://aws.amazon.com/blogs/compute/working-with-events-and-amazon-eventbridge-schema-registry/>
- <https://www.boyney.io/blog/2022-08-09-event-validation>
- <https://aws.plainenglish.io/event-driven-solution-on-aws-371f47792a20>
- [https://d1.awsstatic.com/events/Summits/reinvent2022/API307-R\\_Designing-event-driven-integrations-using-Amazon-EventBridge.pdf](https://d1.awsstatic.com/events/Summits/reinvent2022/API307-R_Designing-event-driven-integrations-using-Amazon-EventBridge.pdf) (Event Driven Topologies with AWS Services)



## SaaS Low-Code Brokers

When comparing SaaS API integration/automation low-code solutions (such as make.com, n8n.io, and zapier.com) to RabbitMQ, there are significant disadvantages to consider. These SaaS solutions have limitations and costs associated with calling millions of API endpoints, as they come with subscription plans that have usage quotas and limits.

For example, during special load-stress situations million API calls will be made in a short time frame, the costs and scalability of these SaaS solutions become important topics. Even their enterprise subscription plans may not accommodate such high message volumes, potentially leading to significant expenses.

Estimating the number of messages sent during a typical month, excluding special events, is challenging due to unpredictable scenarios like mistakes or random batch message publications.

These factors can further complicate the cost and usage considerations of SaaS low-code brokers. However, it's still important to include these solutions in the comparative analysis to evaluate the full range of available options and assess their pros and cons based on specific requirements and budget considerations.

# Comparison

**Interpretation:** Greener is better (like traffic lights). **Ranking:** It is the preferred solution for companies using RMQ considering current and foreseen technology evolution in the next 5 years (2023-2028). The table is constructed based on facts collected in this document.

| Aspect                  | RabbitMQ (AWS EC2) | AWS MQ for RMQ   | Apache Kafka | AWS MS for Kafka | Apache Camel | Apache Pulsar     | AWS SQS (+SNS)    | AWS Event Bridge  | Low-Code Brokers             |
|-------------------------|--------------------|------------------|--------------|------------------|--------------|-------------------|-------------------|-------------------|------------------------------|
| Service Type            | IaaS               | PaaS             | IaaS         | PaaS             | IaaS         | IaaS              | PaaS              | PaaS              | SaaS                         |
| Initial Rel. (Maturity) | 2007 (16yr)        | 2020 (3yr)       | 2011 (12yr)  | 2018 (5yr)       | 2007 (16yr)  | 2019 (4yr)        | 2006 (17yr)       | 2019 (4yr)        | 202x (<3yr)                  |
| Familiarity             | High               | High             | None         | Medium           | None         | None              | Medium            | None              | Low                          |
| Adoption                | High               | Low              | None         | Low              | None         | None              | Low               | None              | Low                          |
| Servers Effort          | Medium             | None             | High         | Low              | High         | High              | None              | None              | None                         |
| Cluster Effort          | High               | Low              | High         | Low              | High         | High              | None              | None              | None                         |
| Message Replay          | No                 | No               | Yes          | Yes              | No           | Yes               | No                | Yes               | N/A                          |
| Message Transform       | No                 | No               | Yes          | Yes              | Yes          | Yes               | No                | Yes               | N/A                          |
| Quotas / Limits         | Soft               | Hard             | Soft         | Hard             | Soft         | Soft              | Hard              | Hard              | Hard                         |
| Defensive Side          | Client             | Client           | Client       | Client           | Client       | System (Throttle) | System (Throttle) | System (Throttle) | System (Throttle)            |
| Code / Logic Refactor   | None               | None             | High         | High             | High         | High              | High              | High              | High                         |
| Scale Up (Vertical)     | Unlimited          | Limited          | Unlimited    | Limited          | Limited      | Unlimited         | Limited           | Limited           | Limited                      |
| Scale Out (Horizontal)  | Very Limited       | Limited          | Unlimited    | Limited          | Very Limited | Unlimited         | Unlimited         | Unlimited         | Limited                      |
| Cloud Vendor Lock-In    | None               | Partial          | None         | Partial          | None         | None              | Yes               | Yes               | Yes                          |
| Monitoring / Alerts     | Prometheus         | AWS CW (Limited) | Prometheus   | AWS CW (Limited) | Prometheus   | Prometheus        | AWS CW            | AWS CW            | Proprietary (Limited)        |
| Certified Support       | High (Contract)    | High (AWS)       | Low          | High (AWS)       | Low          | Medium (Startups) | High (AWS)        | High (AWS)        | High (3 <sup>rd</sup> Party) |
| Ranking                 | 1                  | 3                | 6            | 5                | 8            | 4                 | 2                 |                   | 7                            |

# Conclusions

No matter what companies decide, either **continuing with RMQ** or **switching to another alternative**, it is important to **refactor application logic or implement code fixes** for the issues described in the well-known trade offs, some of them are blocking improvements on RMQ server side and are needed to improve MQ system status quo to be ready for the next challenges.

Otherwise, from all the possible RMQ alternatives, the **AWS Event Bridge / SQS / SNS** set of service makes the most sense instead of maintaining clusters and servers, if the followings are disregarded: required migration effort, additional AWS IAM complexity, novelty of several features and the poor observability based on AWS CloudTrail / CloudWatch.

Maintaining a **MQ system dual stack** for transition is complex and expensive, so an accelerated but long term project plan is required, because replication of messages is key to avoid downtime (Big Bang is not possible).

The **majority of the effort** (and control) will be on the **developer side**, since AWS Event Bridge / SNS / SQS are serverless, RMQ architects/administrators can help define the standards to prevent bottlenecks but AWS is already diligent and very restrictive on what is possible or not.

Finally, any **transition project to AWS** should consider the following major tasks:

- Certify the new AWS-based architecture (single or multi-account).
- Map RMQ routing (bindings) keys to AWS topic-subscription model.
- Define the environments (staging, integration?, production) to be considered.
- Define the naming, filter rules, routing rules (translation?), security rules and terraform conventions.
- Define the minimum observability and recommended alerting standards for key resources in AWS CloudWatch.
- Define the schema real-time schema validation strategy that has to be coded in consumers/publishers (on start time).
- Run a pilot for the high performance bridge between RMQ and AWS Event Bridge (if needed):
  - Federation + Bridge: RMQ => AWS MQ for RabbitMQ => AWS Event Bridge Pipeline / Event Bus
  - In-House Bridge: RMQ => In-House Bridge => SNS/SQS => AWS Event Bridge Pipeline / Event Bus.
- Narrow the magnitude of the monthly costs for the required AWS resources (See AWS Cost Estimation section).
- Careful attention must be paid in consumer/publisher about credential caching versus AWS IAM throttling.

References:

- [https://d1.awsstatic.com/events/Summits/reinvent2022/API307-R\\_Designing-event-driven-integrations-using-AWS-EventBridge.pdf](https://d1.awsstatic.com/events/Summits/reinvent2022/API307-R_Designing-event-driven-integrations-using-AWS-EventBridge.pdf) (Event Driven Topologies with AWS Services)

# Cost Estimation (AWS)

The table below show the additional cost of the AWS Event Bridge reference architecture previously explained:

- Based on 64KB messages, for 256KB messages just multiply 4x the requests.
- Includes the traffic produced by ONE (1) AWS RabbitMQ cluster (excluding logging).
- Based on the total amount messages processed: 600 millions/month (or 231/second).
- Excluding the cost of the related AWS services used by the AWS RabbitMQ cluster itself.
- Based on a single AWS region architecture (this translates into less data transfer costs).

| AWS Service                                                                                                                                                                                                                                                                              | Usage                                                                  | Layer                                                 | Price                                        | Demand                               | Cost (Monthly) |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------|--------------------------------------|----------------|
| Amazon MQ                                                                                                                                                                                                                                                                                | RabbitMQ and Event Bridge integration (with Federation).               | On-Premise Federation for Transition                  | \$2.304/hr mq.m5.4xlarge                     | 3 instances                          | <b>\$5046</b>  |
| Pipes (Pipeline)                                                                                                                                                                                                                                                                         | Source/Targeting AWS Services and 3rd Parties. Filtering capabilities. | Bridge between Amazon MQ and Event Bus for Transition | \$0.4/mio request                            | 600 mio/mo                           | <b>\$240</b>   |
| Event Bus                                                                                                                                                                                                                                                                                | Cross Account Routing to SNS/SQS.                                      | Global Distribution / Publishing                      | \$1 / mio. events                            | 600 mio/mo                           | <b>\$600</b>   |
| Scheduler                                                                                                                                                                                                                                                                                | Scheduled Events (AWS Default Event Bus Only)                          | Custom Scheduling                                     | \$1 / mio. events                            | Not used.                            | -              |
| API Destinations                                                                                                                                                                                                                                                                         | External API calls                                                     | Integration                                           | \$0.20 / mio. request                        | Not used.                            | -              |
| Event Replay                                                                                                                                                                                                                                                                             | Archive Processing                                                     | Republishing                                          | \$0.10/GB                                    | 600 mio/mo * 64KB = 34800 GB / month | <b>\$3480</b>  |
|                                                                                                                                                                                                                                                                                          | Archive Space                                                          |                                                       | \$0.023/GB/mo                                | Same as above but only 1 week.       | <b>\$200</b>   |
| Schema Registry                                                                                                                                                                                                                                                                          | Schema validation                                                      | Validation                                            | \$0.10/million events ingested for discovery | Not relevant.                        | <b>\$0</b>     |
| SNS                                                                                                                                                                                                                                                                                      | Basic Routing for SQS or any other AWS services.                       | Local Distribution / Publishing                       | No charge for SQS deliveries                 | 600 mio/mo                           | <b>\$0</b>     |
|                                                                                                                                                                                                                                                                                          |                                                                        |                                                       | \$0.085/GB Data Transfer (Out)               | 34800 GB                             | <b>\$2958</b>  |
| SQS                                                                                                                                                                                                                                                                                      | Queue                                                                  | Consumption                                           | \$0.40 mio (Std)<br>-<br>\$0.50 mio (FIFO)   | 600 mio/mo                           | <b>\$240</b>   |
|                                                                                                                                                                                                                                                                                          |                                                                        |                                                       | \$0.085/GB Data Transfer (Out)               | 34800 GB                             | <b>\$2958</b>  |
| Additional Total (Transition)                                                                                                                                                                                                                                                            |                                                                        |                                                       |                                              |                                      | <b>\$15722</b> |
| Additional Total (Final)                                                                                                                                                                                                                                                                 |                                                                        |                                                       |                                              |                                      | <b>\$10436</b> |
| <b>NOTE:</b> AWS services offer a pay-as-you-go pricing model, it is important to emphasise that the costs can go wild based on the number of messages, API requests/calls, and additional AWS features used. So, any mistake will cost a lot of money as there is no way of contain it. |                                                                        |                                                       |                                              |                                      |                |